H5: Loops deel 1

Labo 2 · week 5 · complexe condities, oneindige loops en scope

SEMESTER 1 · BASIS PROGRAMMEREN

Vandaag

1. **Opwarmen**: `while` of `do while`?
2. **Complexe condities**: loops met `&&` en `||`
3. **Oneindige loops**: bewust of per ongeluk
4. **Scope in loops**: waar declareer je je variabelen?
5. **Aan de slag** met de rest van de while-oefeningen



Vandaag: oneindige loops. We stoppen wel op tijd. Beloofd. Waarschijnlijk.

Opwarmen

`while` of `do while`? 🤔

Welke loop past het best?

1. Een menu tonen tot de gebruiker “stoppen” kiest → **`do while`**
2. De getallen van 1 tot 10 tonen → **`while`**
3. Een wachtwoord vragen tot het juist is → **`do while`**
4. Woorden inlezen tot de gebruiker “stop” typt, ook als hij meteen “stop” typt → **`while`**

`do while` als het **minstens één keer** moet gebeuren, anders `while`.

 *`do while` is als een tweede kans bij een examen: eerst afleggen, dan pas beslissen of het opnieuw moet.*

Complexe condities

Loops met `||`

De voorwaarde van een loop is een booleaanse expressie, dus ook `&&`, `||` en `!` mogen:

```
 1  Console.WriteLine("Ben je ziek? (ja/nee)");
 2  bool isSick = Console.ReadLine() == "ja";
 3  Console.WriteLine("Ben je moe? (ja/nee)");
 4  bool isTired = Console.ReadLine() == "ja";
 5  while (isSick || isTired)
 6  {
 7      Console.WriteLine("Blijf in bed!");
 8      Console.WriteLine("Ben je ziek? (ja/nee)");
 9      isSick = Console.ReadLine() == "ja";
10      Console.WriteLine("Ben je moe? (ja/nee)");
11      isTired = Console.ReadLine() == "ja";
12  }
13  Console.WriteLine("Opstaan!");
```

Blijf in bed zolang je ziek of moe bent. Voor sommige studenten een oneindige loop.

Twee redenen om te stoppen

Maximaal **drie pogingen** voor het juiste wachtwoord:

```
1 int attempts = 0;
2 string password = "";
3 while (password != "gEhEiM" && attempts < 3)
4 {
5     Console.WriteLine("Wachtwoord?");
6     password = Console.ReadLine();
7     attempts++;
8 }
```

Met `&&` gaat de loop door zolang **alles** klopt. Hij stopt zodra **één** van de voorwaarden `false` wordt: het wachtwoord is juist, **of** de pogingen zijn op.

Drie keer fout? Geblokkeerd. Net als je bankapp om drie uur 's nachts.

Waarom stopte de loop?

Na de loop weet je niet **waarom** hij stopte. Controleer het met een `if`:

```
1  if (password == "gEhEiM")
2  {
3      Console.WriteLine($"Welkom! Gelukt na {attempts} pogingen.");
4  }
5  else
6  {
7      Console.WriteLine("Geblokkeerd.");
8  }
```

```
Wachtwoord?
fout
Wachtwoord?
gEhEiM
Welkom! Gelukt na 2 pogingen.
```



De loop zegt niet waarom hij stopte. Net als een collega die zonder uitleg naar huis vertrekt.

Hoe vaak? 🤔

```
1  int count = 0;
2  int total = 0;
3  while (count < 10 && total < 20)
4  {
5      count++;
6      total = total + count;
7  }
8  Console.WriteLine($"{count} {total}");
```

```
6 21
```

`total` wordt 1, 3, 6, 10, 15 en dan 21. Bij 21 is `total < 20` `false`, dus de loop stopt, ook al is `count` nog maar 6.

21: het magische getal bij blackjack. Hier gewoon het einde van een loop.

Oneindige loops

Bewust oneindig

```
1 while (true)
2 {
3     // code die nooit hoort te stoppen
4 }
```

Soms is dat **gewenst**: een game die steeds het scherm ververst, of een programma dat blijft wachten op invoer.

> **TIP**
>
> Een programma in de console stoppen? **Ctrl + C**, of in Visual Studio de rode stopknop (**Shift + F5**).

`while (true)`: de loop die gelooft dat alles altijd waar is. Een optimist.

Per ongeluk oneindig

```
1  int counter = 0;
2  while (counter < 10)
3  {
4      Console.WriteLine(counter);
5  }
```

`counter` blijft **altijd 0**. De voorwaarde wordt nooit `false`, dus de loop stopt nooit.

Oplossing: verander in de loop iets dat de voorwaarde beïnvloedt, hier `counter++;`.

😈 *Een loop zonder `counter++` is als een hamster in een rad: veel beweging, geen vooruitgang.*

De verraderlijke puntkomma

```
1  int count = 0;
2  while (count < 10);
3  {
4      Console.WriteLine(count);
5      count++;
6  }
```

De `;` achter `while (...)` is een **lege opdracht**. Die wordt eindeloos herhaald, en het blok met accolades komt nooit aan de beurt.

> ⚠ **WAARSCHUWING**
>
> Dit compileert gewoon! Visual Studio geeft enkel een **waarschuwing**: CS0642 “Possible mistaken empty statement”. Lees dus ook de groene kronkellijnen.

Eén puntkomma te veel, en je programma doet urenlang… niets. Heel efficiënt niets.

Checklist bij elke loop

1. Heeft de voorwaarde een **beginwaarde** om mee te starten?
2. Verandert er **in** de loop iets wat de voorwaarde beïnvloedt?
3. Gaat die verandering in de **goede richting**, naar `false` toe?
4. Staat er **geen** puntkomma achter `while (...)`?

 *Ctrl + C: de noodrem van de console. Geen boete voor misbruik.*

Scope in loops

De som van 1 tot 10, fout

```
1  int counter = 1;
2  while (counter <= 10)
3  {
4      int sum = 0;
5      sum = sum + counter;
6      counter++;
7  }
8  Console.WriteLine(sum);  // ❌ CS0103
```

Twee problemen:

1. `sum` bestaat enkel **binnen** de accolades, dus na de loop niet meer.
2. `sum` wordt in **elke ronde** opnieuw aangemaakt en op 0 gezet.

 *`sum` begint elke ronde opnieuw bij 0. Zoals je spaarrekening na de solden.*

De som van 1 tot 10, juist

```
1  int counter = 1;
2  int sum = 0;
3  while (counter <= 10)
4  {
5      sum = sum + counter;
6      counter++;
7  }
8  Console.WriteLine(sum);  // 55
```

`sum` wordt **één keer** aangemaakt, vóór de loop. Elke ronde telt er iets bij, en na de loop kan je het resultaat tonen.

Gauss telde als kind 1 tot 100 op in zijn hoofd. Wij gebruiken een loop. En koffie.

Vuistregel

Wat?	Waar declareren?
moet **over de rondes heen** bewaard blijven (teller, som, resultaat)	**vóór** de loop
enkel nodig **binnen één ronde** (een tussenresultaat)	mag **in** de loop
nodig in de **voorwaarde** van een `do while`	**vóór** de loop

Wat je wil onthouden, hoort buiten de loop. Net als je sleutels: niet in de wasmachine.

Samengevat

Wat	Onthoud
complexe conditie	de voorwaarde zegt wanneer je **doorgaat**
na de loop	controleer met een `if` **waarom** hij stopte
oneindige loop	verander in de loop iets wat de voorwaarde beïnvloedt
puntkomma	**geen** `;` achter `while (...)`, wel achter `do { } while (...);`
scope	wat over rondes heen bewaard moet blijven: **vóór** de loop

Je weet nu hoe je een loop laat stoppen. Dat is meer dan de meeste vergaderingen.

Aan de slag 💻

Maak verder in de klasse `Loops`:

1. **Party**: schrijf personen in, maximaal 20, en toon op het einde alle namen
2. **SumOfEvenNumbers**: de som van alle even getallen tot n, met tussensommen
3. **Factors**: toon alle factoren van een getal
4. **RNATranscription**: zet DNA-letters één voor één om naar RNA
5. **Accountant**: een boekhoudprogramma dat blijft doorgaan

👉 Labo 1 nog niet af? Werk dan eerst CountDown, Password, Average, Lions en NumberOfDigits af.

😈 `while (!partyOver) { Dance(); }`